



K.R. Mangalam University
School of Engineering & Technology

Fundamentals Of Java Programming Lab
(ENCA203) Assignment 3
Student Management System

Submitted by:

Name: Shivam Gaur

Roll no. 2401201016

Class: BCA (h) (AI & DS)

Submitted to:

Dr. Manish Kumar

GitHub Repository:

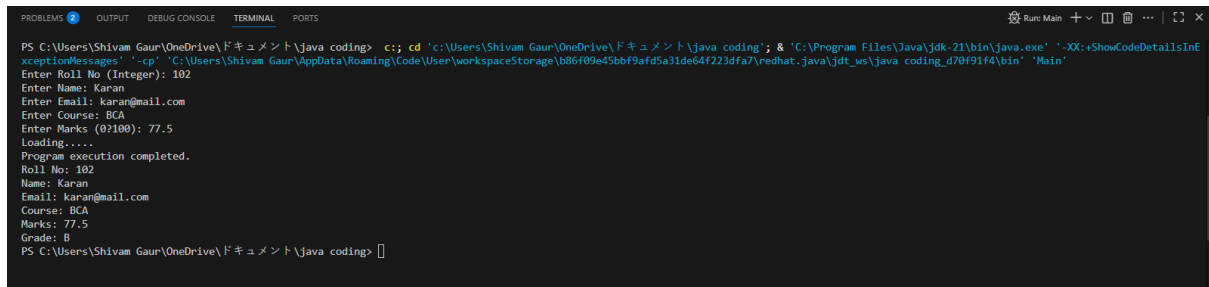
https://github.com/shivamgaur1255-sketch/java_assignments-

Codes:

```
J Calculator.java 1 J Main.java 1
J Main.java > Loader > run()
1 import java.util.Scanner;
2
3 // Custom Exception
4 class StudentNotFoundException extends Exception {
5     public StudentNotFoundException(String message) {
6         super(message);
7     }
8 }
9
10 // Loader class for multithreading
11 class Loader implements Runnable {
12     @Override
13     public void run() {
14         try {
15             System.out.print(s:"Loading");
16             for (int i = 0; i < 5; i++) {
17                 Thread.sleep(millis:500);
18                 System.out.print(s:".");
19             }
20             System.out.println();
21         } catch (InterruptedException e) {
22             System.out.println(x:"Loading Interrupted!");
23         }
24     }
25 }
26
27 // Student Class
28 class Student {
29     Integer rollNo;
30     String name;
31     String email;
32     String course;
33     Double marks;
34
35     public Student(Integer rollNo, String name, String email, String course, Double marks) {
36         this.rollNo = rollNo;
37         this.name = name;
38         this.email = email;
39         this.course = course;
40         this.marks = marks;
41     }
42
43     public String getGrade() {
44         if (marks >= 90) return "A";
45         else if (marks >= 75) return "B";
46         else if (marks >= 60) return "C";
47         else if (marks >= 40) return "D";
48         else return "F";
49     }
50
51     public void display() {
52         System.out.println("Roll No: " + rollNo);
53         System.out.println("Name: " + name);
54         System.out.println("Email: " + email);
55         System.out.println("Course: " + course);
56         System.out.println("Marks: " + marks);
57         System.out.println("Grade: " + getGrade());
58     }
59 }
60
61 // Student Manager Class
62 class StudentManager {
63
64     Student student;
65
66     public void addStudent() {
67         Scanner sc = new Scanner(System.in);
68
69         try {
70             System.out.print(s:"Enter Roll No (Integer): ");
71             Integer roll = Integer.parseInt(sc.nextLine());
72
73             System.out.print(s:"Enter Name: ");
74             String name = sc.nextLine();
75             if (name.isEmpty())
76                 throw new Exception(message:"Name cannot be empty!");
77
78             System.out.print(s:"Enter Email: ");
79             String email = sc.nextLine();
80
81             Student student = new Student(roll, name, email, "", 0.0);
82         } catch (Exception e) {
83             System.out.println(e.getMessage());
84         }
85     }
86
87     public void display() {
88         System.out.println("Student Details:");
89         student.display();
90     }
91
92     public void run() {
93         while (true) {
94             System.out.println("1. Add Student\n2. Display Student\n3. Exit\n");
95             int choice = Integer.parseInt(sc.nextLine());
96
97             switch (choice) {
98                 case 1:
99                     addStudent();
100                 case 2:
101                     display();
102                 case 3:
103                     System.out.println("Exiting...");
104                     break;
105             }
106         }
107     }
108 }
109
110 public class Main {
111     public static void main(String[] args) {
112         StudentManager manager = new StudentManager();
113         manager.run();
114     }
115 }
```

```
J Calculator.java 1 J Main.java 1
J Main.java > Student > display()
62 class StudentManager {
66     public void addStudent() {
76         throw new Exception(message:"Name cannot be empty!");
77
78         System.out.print(s:"Enter Email: ");
79         String email = sc.nextLine();
80         if (email.isEmpty())
81             throw new Exception(message:"Email cannot be empty!");
82
83         System.out.print(s:"Enter Course: ");
84         String course = sc.nextLine();
85         if (course.isEmpty())
86             throw new Exception(message:"Course cannot be empty!");
87
88         System.out.print(s:"Enter Marks (0-100): ");
89         Double marks = Double.parseDouble(sc.nextLine());
90         if (marks < 0 || marks > 100)
91             throw new Exception(message:"Marks must be between 0 and 100!");
92
93         Thread loader = new Thread(new Loader());
94         loader.start();
95         loader.join(); // wait for thread
96
97         student = new Student(roll, name, email, course, marks);
98
99     } catch (NumberFormatException nfe) {
100         System.out.println(x:"Invalid number format! Enter proper numeric values.");
101     } catch (Exception e) {
102         System.out.println("Error: " + e.getMessage());
103     } finally {
104         System.out.println(x:"Program execution completed.");
105     }
106 }
107
108 public void displayStudent(int roll) throws StudentNotFoundException {
109     if (student == null || !student.rollNo.equals(roll)) {
110         throw new StudentNotFoundException("Student with Roll No " + roll + " not found!");
111     }
112
113     student.display();
114 }
115 }
116
117 // Main Class
118 public class Main {
119     Run | Debug
120     public static void main(String[] args) {
121         StudentManager sm = new StudentManager();
122         sm.addStudent();
123
124         try {
125             sm.displayStudent(roll:102); // Example roll number
126         } catch (StudentNotFoundException e) {
127             System.out.println(e.getMessage());
128         }
129     }
130 }
131
132
```

Output:



```
PS C:\Users\Shivam Gaur\OneDrive\ドキュメント\java coding> c:: cd 'C:\Users\Shivam Gaur\OneDrive\ドキュメント\java coding'; & 'C:\Program Files\Java\jdk-21\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Shivam Gaur\AppData\Roaming\Code\User\workspaceStorage\b86f09e45bbf9afd5a31de64f223dfa7\redhat.java\jdt_ws\java coding_d70f91f4\bin' 'Main'
Enter Roll No (Integer): 102
Enter Name: Karan
Enter Email: karan@mail.com
Enter Course: BCA
Enter Marks (0:100): 77.5
Loading.....
Program execution completed.
Roll No: 102
Name: Karan
Email: karan@mail.com
Course: BCA
Marks: 77.5
Grade: B
PS C:\Users\Shivam Gaur\OneDrive\ドキュメント\java coding>
```

Explanation:

This Java program is a Student Management System designed to demonstrate exception handling, multithreading, and wrapper classes. It allows the user to add and display student details while ensuring that invalid inputs are handled safely. The program consists of several classes that work together to provide a complete, robust solution.

The `StudentNotFoundException` class is a custom exception that is used to handle situations where a student with a given roll number does not exist. The `RecordActions` interface defines two essential methods, `addStudent()` and `displayStudent(int rollNo)`, which must be implemented by any class that manages student records. The `Loader` class implements the `Runnable` interface and is used to simulate a loading process using multithreading. When a student is added, a separate thread runs to print “Loading.....” with a delay, making the system appear more interactive and realistic.

The main functionality resides in the `StudentManager` class, which implements the `RecordActions` interface. Inside it, there is an inner `Student` class that holds student details such as roll number, name, email, course, and marks. Here, wrapper classes like `Integer` and `Double` are used instead of primitive types to demonstrate autoboxing. The `addStudent()` method collects user input and validates it using `trycatch-finally` blocks. If the user enters invalid data, such as

marks outside the range of 0–100 or empty fields, the program throws an appropriate exception and displays a clear error message. Once validation succeeds, a loader thread runs before saving the data to simulate a delay. All student data is stored in a HashMap using the roll number as a key for quick access.

The `displayStudent()` method retrieves a student's details from the map using their roll number. If the student is not found, it throws the custom `StudentNotFoundException`. The `StudentManagementSystem` class contains the main method, which presents a menu-driven interface that allows users to add students, display records, or exit the program. The menu continuously runs until the user chooses to exit, ensuring easy interaction.